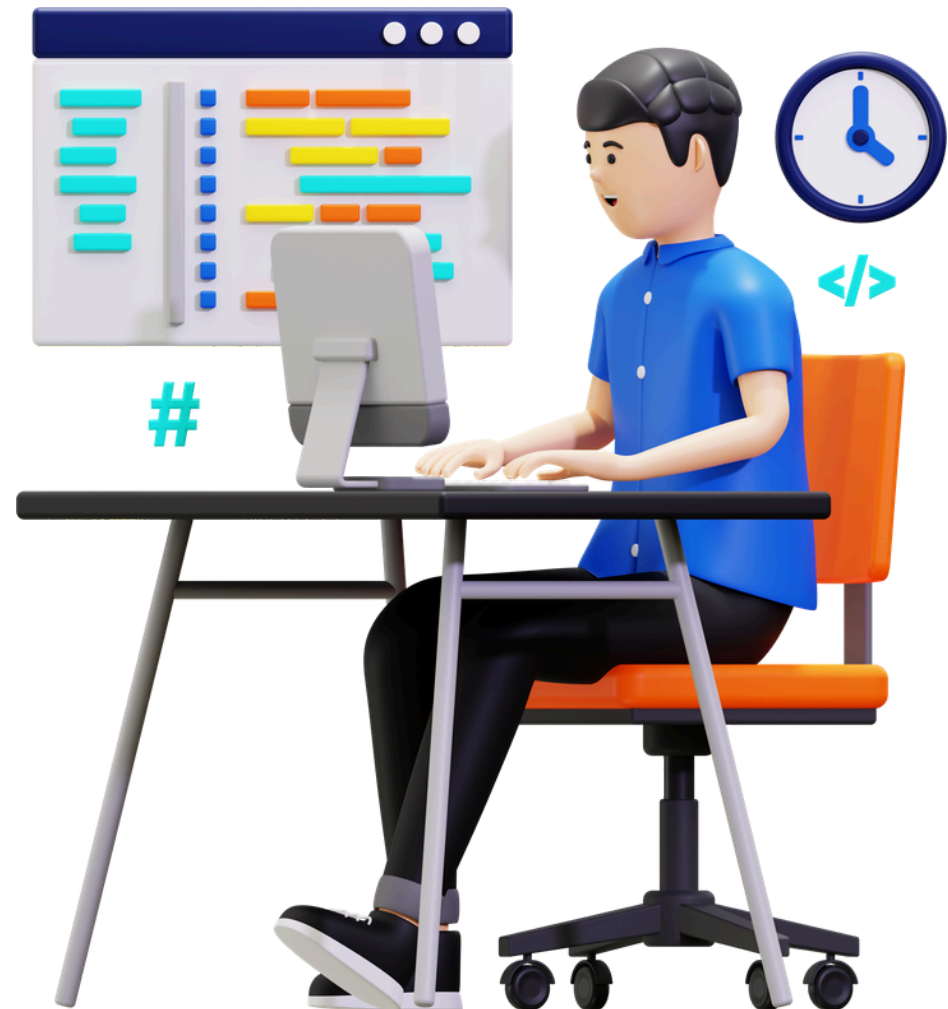
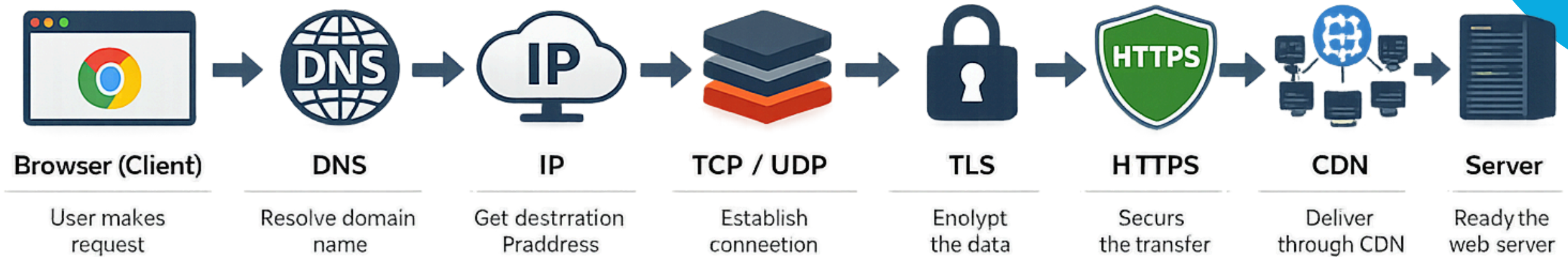


Network Engineering

Q.E.D. — Web Dev Bootcamp | Session 02

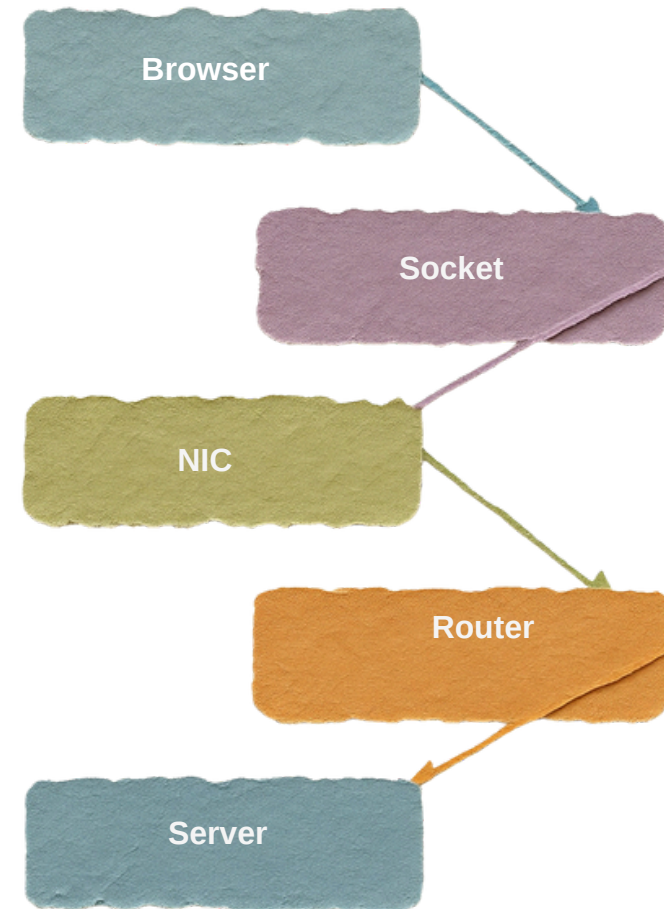


A journey with your request



Almost every internet application communication using TCP or UDP
can be represented as:
source ip/port → destination ip/port

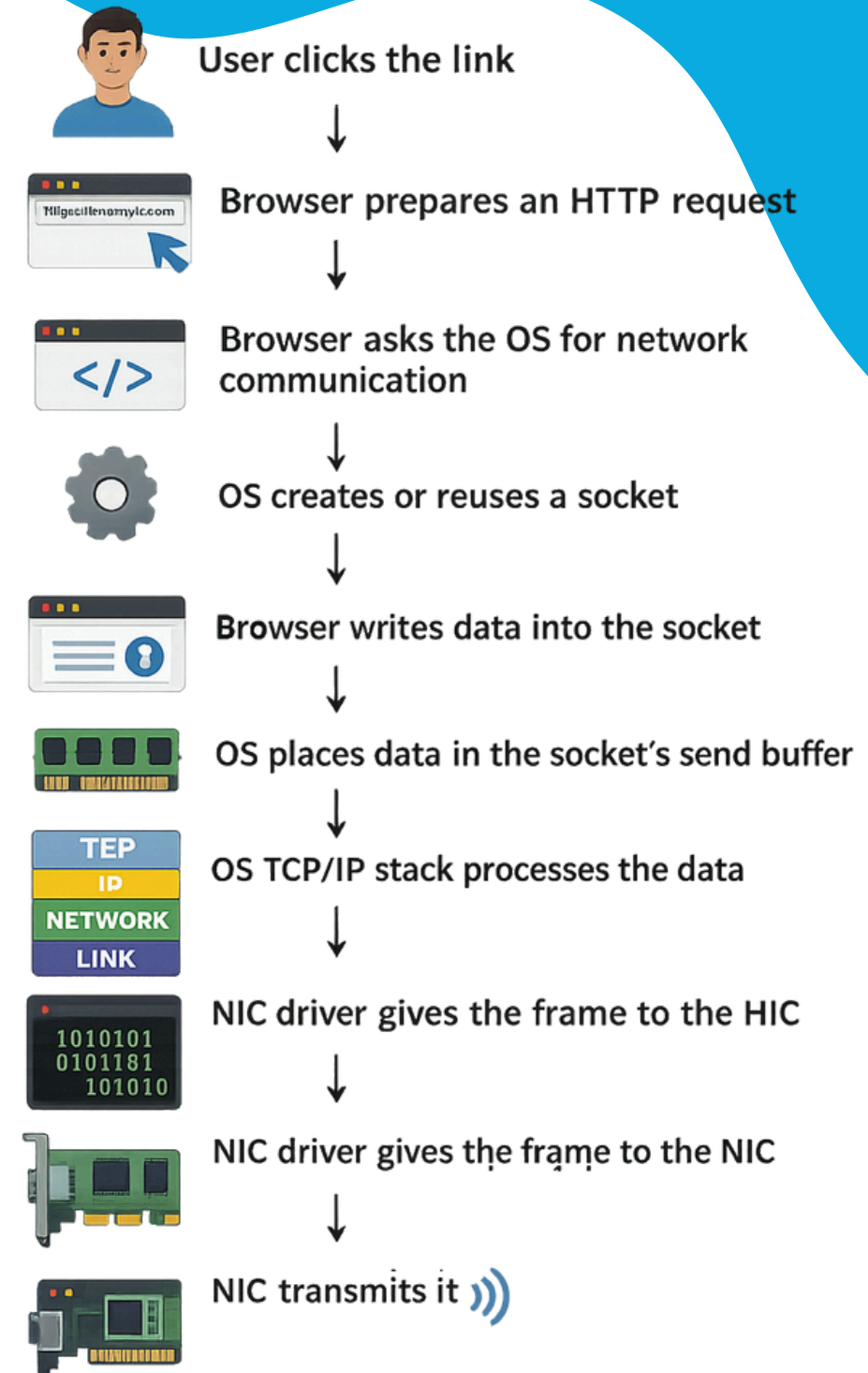
A beginner-friendly journey
through the OS, the
network, and modern
backend architecture.



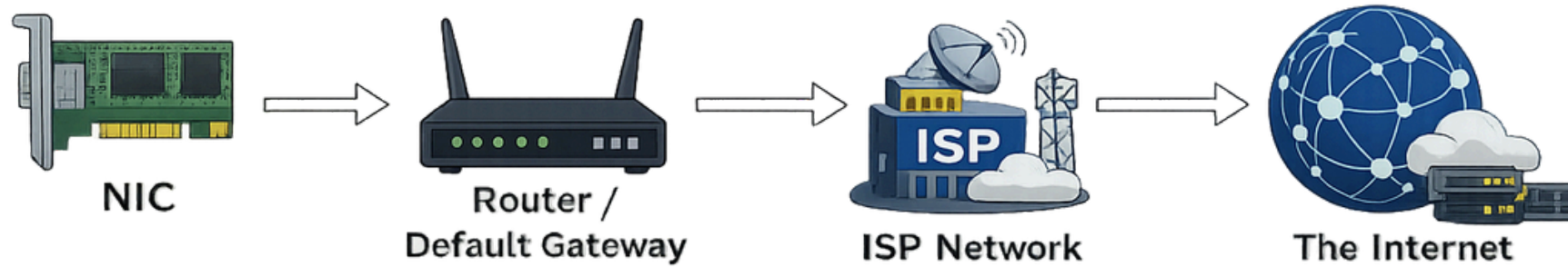
Different Dive to same request

- What is actually a Socket?

A software endpoint managed by the OS kernel. It holds connection identity, protocol state, a send buffer, and a receive buffer. Many sockets can share one NIC.



Different Dive to same request

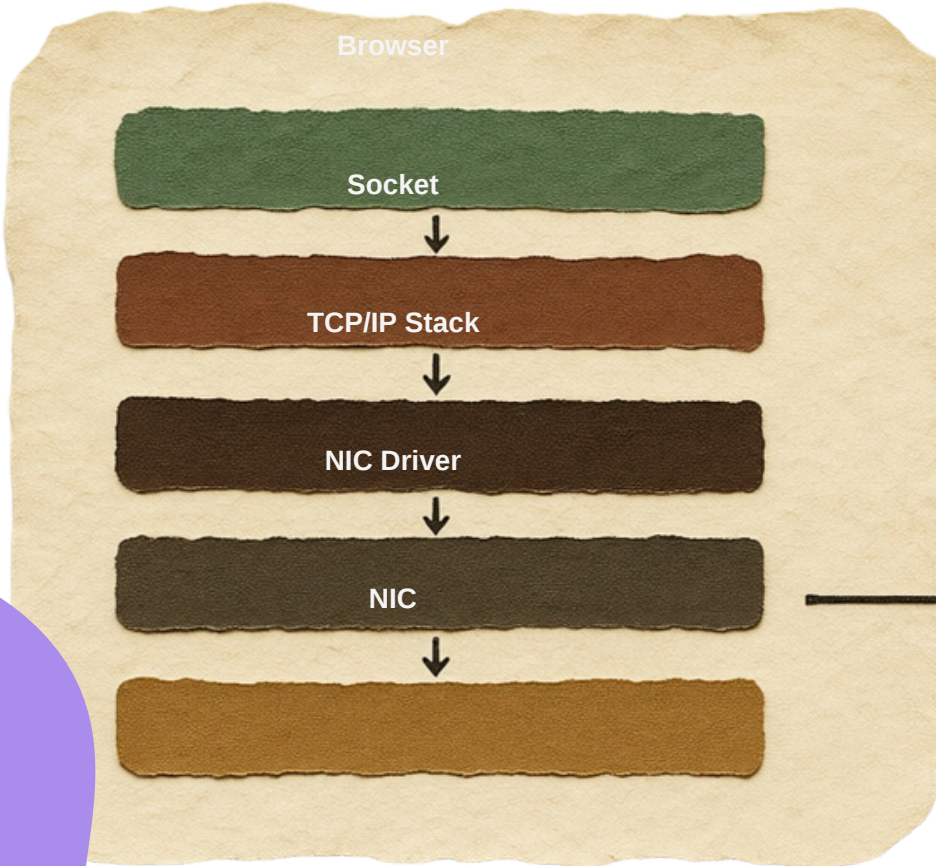


- **What is NIC?**

A physical or virtual network interface. It sends and receives frames, has a link-layer identity, and converts bits into radio, electricity, or light.

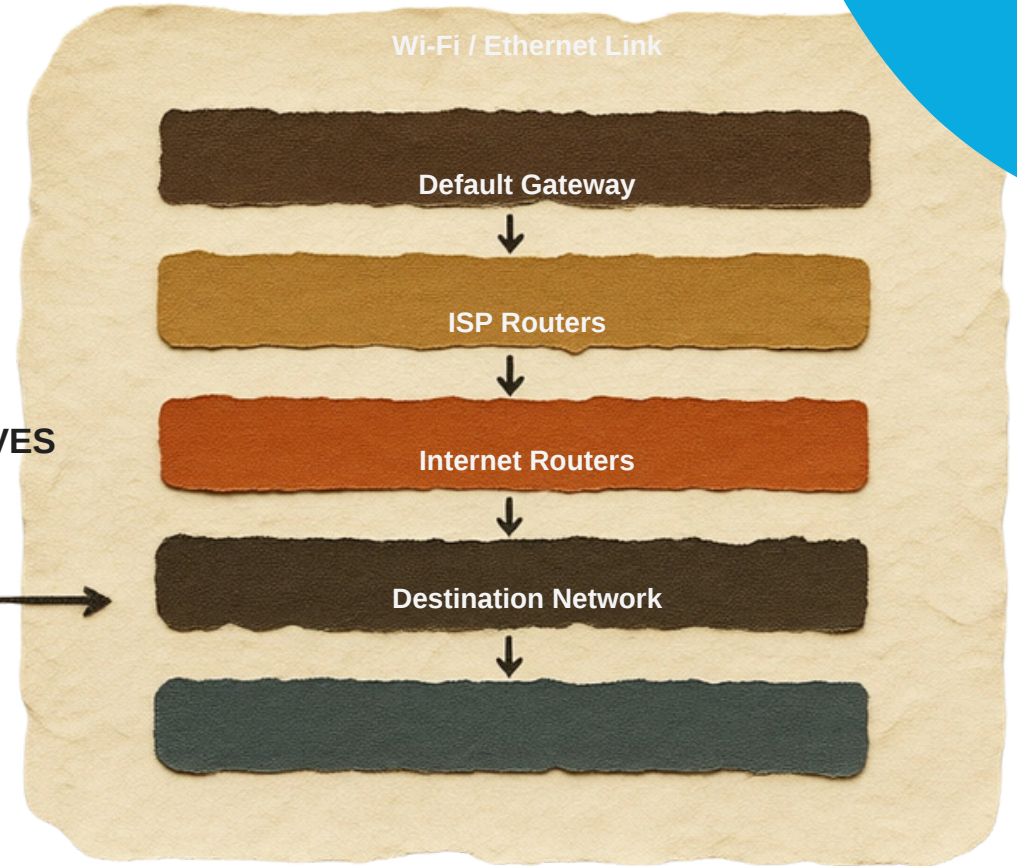
Different Dive to same request

INSIDE THE DEVICE



OUTSIDE ON THE NETWORK

FRAME LEAVES



The six sentences to remember

Keep these distinctions clear and the whole topic becomes easier.

1 The socket connects an application to the OS networking stack.

2 The NIC connects the computer to the physical network.

3 The network forwards packets hop by hop through routers and providers.

4 Client–Server says who asks and who provides the service.

5 Monolith vs Microservices says how the server side is organized.

6 Sync waits for an answer; async publishes and continues.

I Browser → Socket → TCP/IP → NIC → Router → ISP → Internet → Destination Network → Server Socket → Backend

Client-Server Architecture

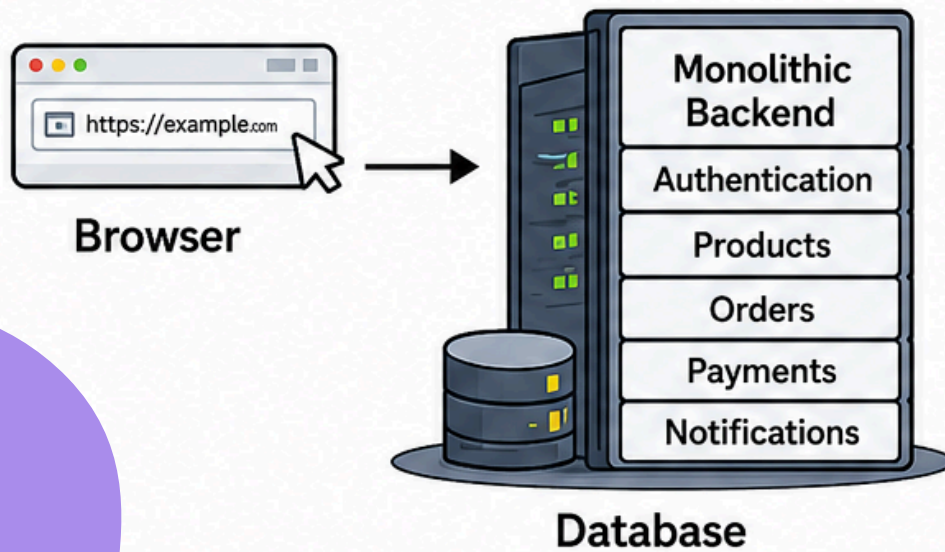
We need a Standard communication models / protocols to handle a lot of problems

- Your app would need to know about every physical medium
- Network hardware from different vendors couldn't talk to each other
- You couldn't improve one part without breaking everything else

Compare Architectures

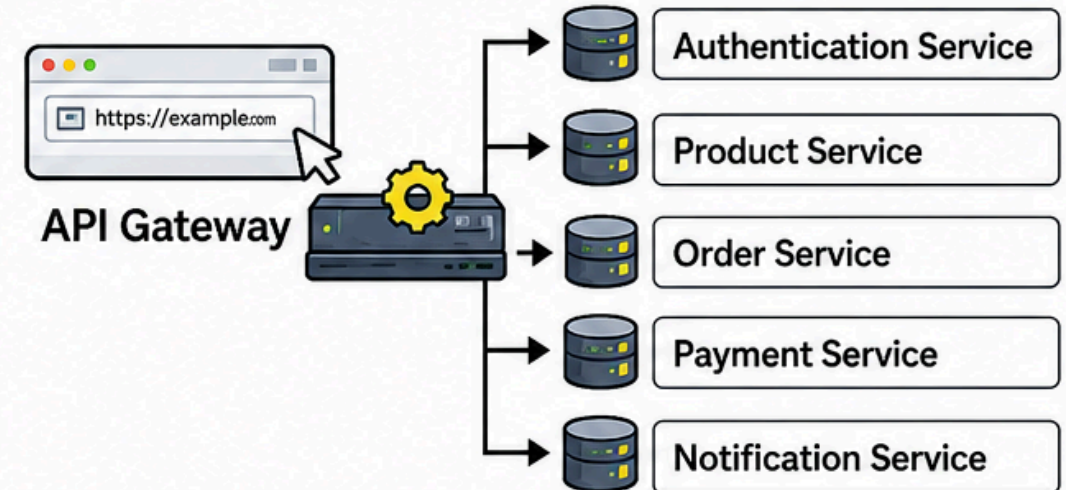
Monolithic Architecture

All features in one application

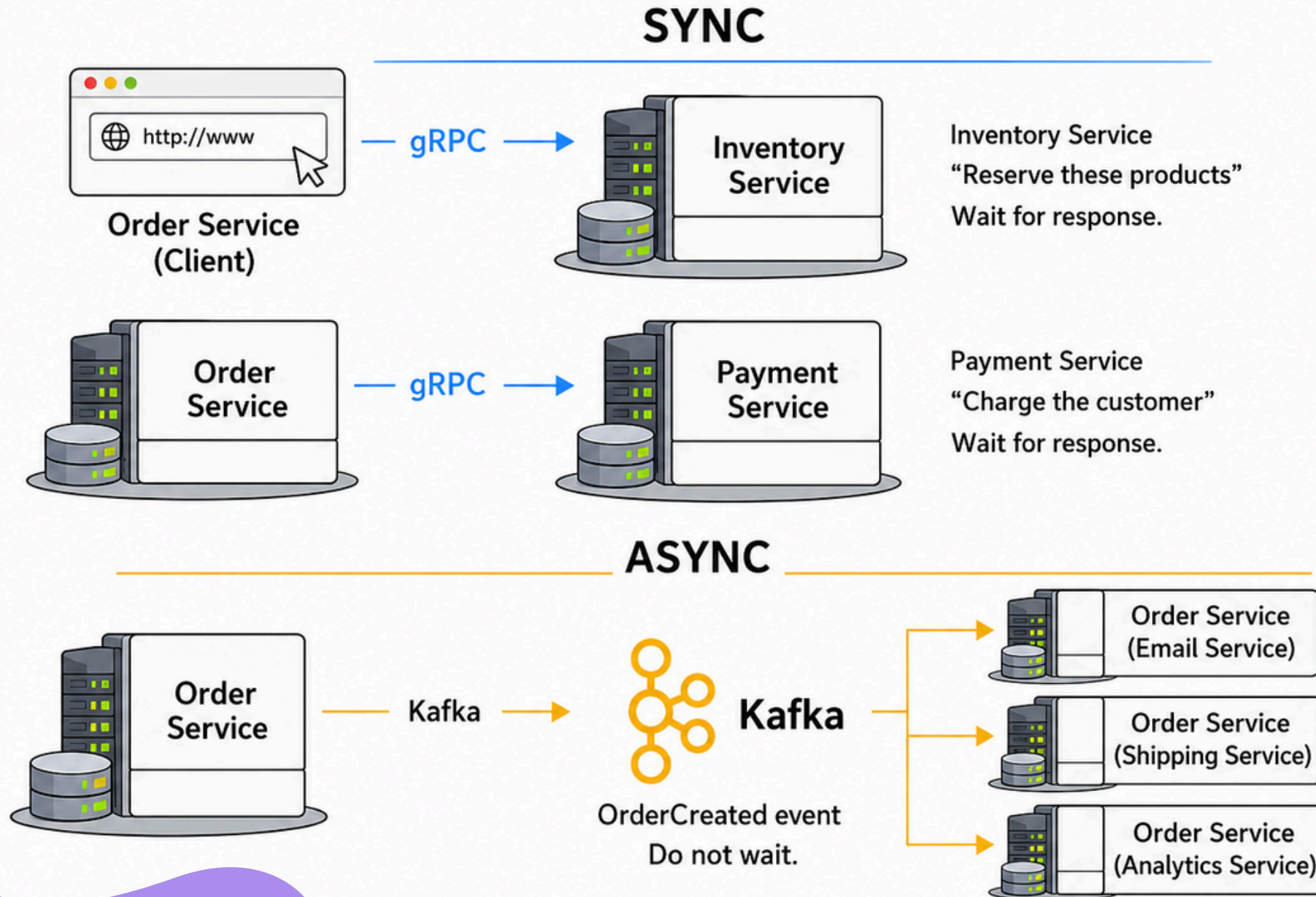


Microservices Architecture

Features split between Independent services

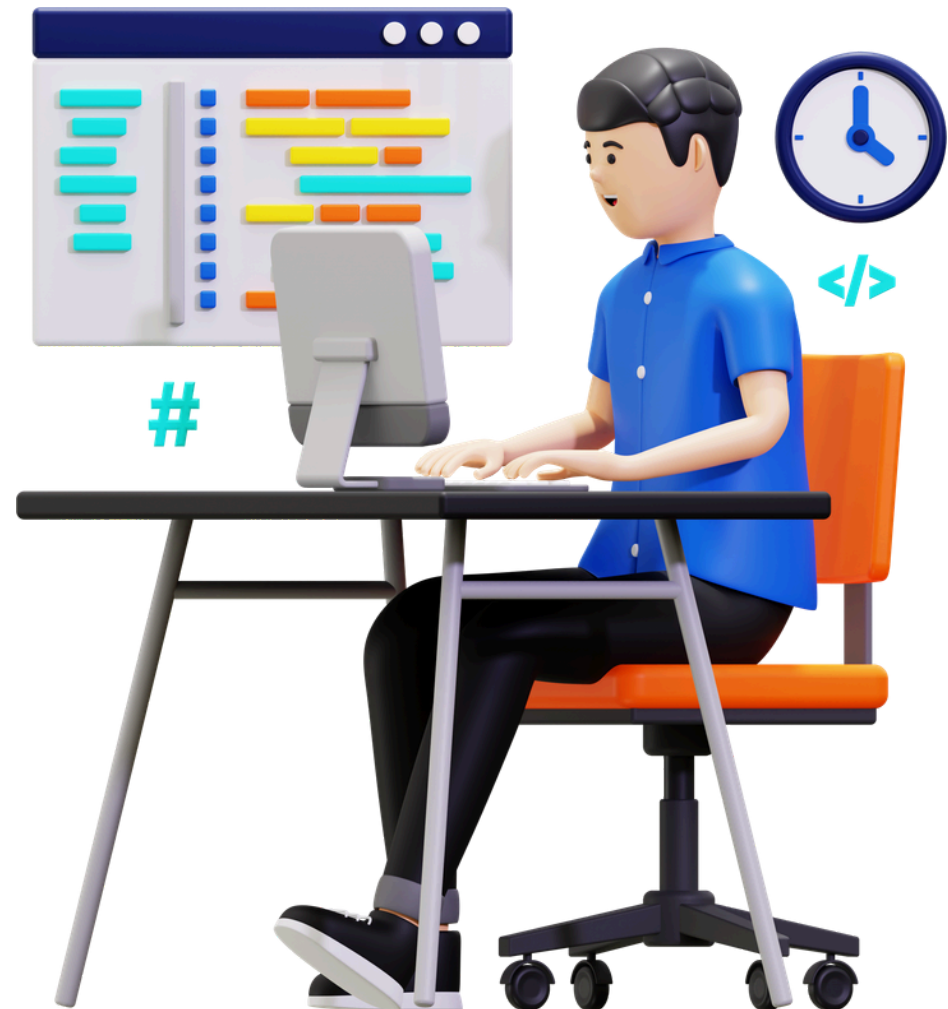


Communication inside the microservices



Fast Api

Q.E.D. — Web Dev Bootcamp | Session 02



1. When to use Path vs Query Parameters?

Feature	Path Parameters	Query Parameters
Purpose	Identify a specific resource.	Filter, sort, or paginate a list.
URL Syntax	/packages/{id}	/packages?city=Cairo&limit=5
Requirement	Required by default.	Optional (if = None is given).

Scenario: A user clicks "Track Package #405". The app needs the exact details of that single item.
Code: @app.get("/packages/{id}")
URL: GET /packages/405

Scenario: Admin opens the dashboard: "Show me the first 10 pending deliveries in Cairo".
Code: def list(city: str, status: str):
URL: GET /packages?city=Cairo&status=pending

2. The Two Layers of Validation

Layer	1. Pydantic Models	2. Business Logic (Your Code)
What it checks	Data Types, Constraints (Int, String).	Real-world rules (State Machines).
Execution	Runs automatically before your code.	Runs manually inside your endpoint.
Error Thrown	422 Unprocessable Entity	400 Bad Request

Scenario: Frontend sends "weight": "heavy" instead of an integer number.
Action: Pydantic blocks it instantly. Your endpoint code never even executes.
Result: Returns **422**.

Scenario: Courier tries to cancel a package that is already marked as "Delivered".
Action: Data types are correct, so Pydantic allows it. Your code catches the logic flaw.
Result: You raise **400**.

3. HTTP Methods in the Real World

Method	Action	Idempotent?	Real-World SwiftDrop Situation
POST	Create	No	E-commerce system pushes a new delivery order to your API.
GET	Read	Yes	Customer refreshes the page to check their package status.
PUT	Update	Yes	Replacing an entire courier profile (overwriting old data).
PATCH	Update	Yes	Courier clicks "Picked Up". We only update the status field.
DELETE	Delete	Yes	Admin deletes a test order from the database.

* **Idempotency:** Running GET, PUT, PATCH, or DELETE 10 times yields the same database state as 1 time.

4. Status Codes Cheat Sheet (When to throw what?)

Code	Meaning	Trigger Situation (Scenario)
200	OK	Default success response for fetching or updating a package.
201	Created	You successfully saved a new POST request to the database.
400	Bad Request	Courier tries to skip "In Transit" and jump straight to "Delivered".
401	Unauthorized	The request is missing a valid API Token in the headers.
403	Forbidden	Logged in, but lacks Admin rights to delete a package.
404	Not Found	Driver scans barcode #999, but it doesn't exist in our DB.
422	Unprocessable	Pydantic expected a valid float for weight, got a string.

5. Implementing Responses & Errors in Code

Setting Custom Status Codes

Situation: We need to return exactly 201 for POST.

```
from fastapi import status

@app.post("/packages", status_code=status.
         HTTP_201_CREATED)
def create_package():
    # ... save to database ...
    return {"message": "Created
            successfully"}
```

Throwing Errors Manually

Situation: A non-existent ID was passed in the URL.

```
from fastapi import HTTPException, status

@app.get("/packages/{id}")
def get_package(id: int):
    package = db.get(id)
    if not package:
        raise HTTPException(
            status_code=status.
              HTTP_404_NOT_FOUND,
            detail="Package not found"
        )
    return package
```

6. Quick Checklist for the Task

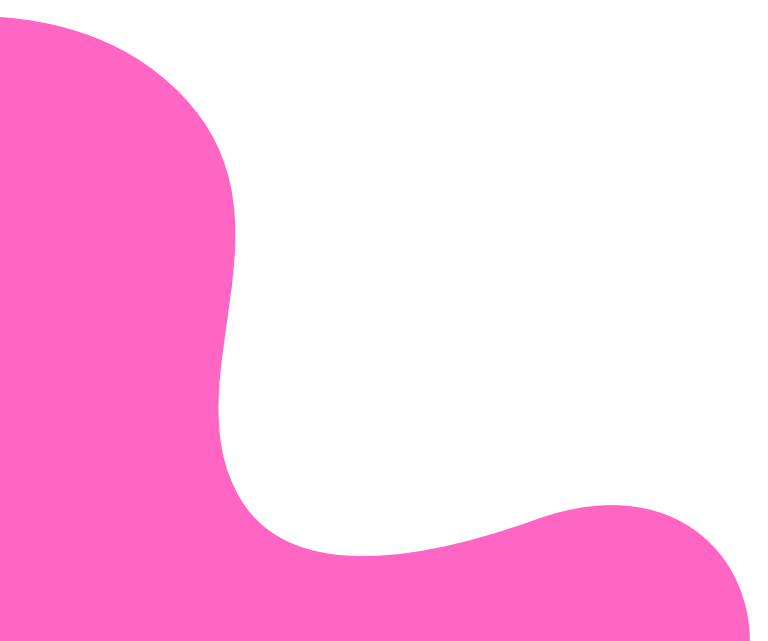
Data Flow Check

- Use **PathParams** to identify a single target.
- Use **QueryParams** to filter lists.
- Let **Pydantic** handle the basic data shape.
- Write your own **Business Rules** (State Machine).

API Compliance Check

- Return **201** for successful POST endpoints.
- Raise **404** if db.get() returns None.
- Raise **400** if an action violates delivery rules.

You're Ready to Task



Thank You